SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE CODE: CSA04
COURSE NAME: Operating Systems

COURSE OUTCOME COVERED

CO1: *Apply the concepts of Operating System types, structures, and system calls to demonstrate their functions in various computing environments.*

QUESTIONS: 5 **Total Marks:~~100~~**

Annexure A

SHORT ANSWER TEST

96/100

Code of Conduct:

I Galla Sai Siddartha (192425093) *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Galla Sai Siddartha

24/25

Annexure A

Short Answer Test

Question 1 (5 Marks)
Suppose a modern Operating System supports multiple users and processes with shared resources like CPU, memory, and I/O devices.
(a) Classify different types of Operating Systems suitable for this scenario and justify your choice.
(b) Draw the structure of an Operating System showing how resources are managed.
(c) Write a sequence of system calls required to create and execute a new process in such an environment. **(BL2)**

Question 2 (5 Marks)
Consider an Operating System managing multiple files and devices for concurrent users.
(a) Categorize system calls used for file and device management.
(b) Illustrate how the OS handles file operations using a layered structure diagram.
(c) Write a pseudo-code or C-based system call sequence to open, read, and close a file. **(BL3)**

Question 3 (5 Marks)
Assume a real-time Operating System used in an embedded system controlling industrial machinery.
(a) Identify the type of OS and explain why it is suitable for real-time applications.
(b) Draw a diagram representing the interaction between hardware, kernel, and user processes.
(c) List and explain system calls required for process synchronization in this system. **(BL2)**

Question 4 (5 Marks)
A distributed computing system consists of multiple interconnected computers sharing resources over a network.
(a) Classify the type of Operating System used and compare it with centralized OS.
(b) Draw the architecture of a distributed OS showing communication between nodes.
(c) Write the sequence of system calls involved in inter-process communication (IPC). **(BL3)**

Question 5 (5 Marks)
Consider a scenario where an Operating System must manage process scheduling, memory allocation, and device communication efficiently.
(a) Explain the major functions of an OS in handling these tasks.
(b) Draw a block diagram representing OS components and their interactions.
(c) Write system call examples for memory allocation and device management. **(BL3)**

1) Types and Structure of Operating Systems

(a) Types of Operating Systems

(i) Multi-user OS - Allows multiple users to access system resources simultaneously.

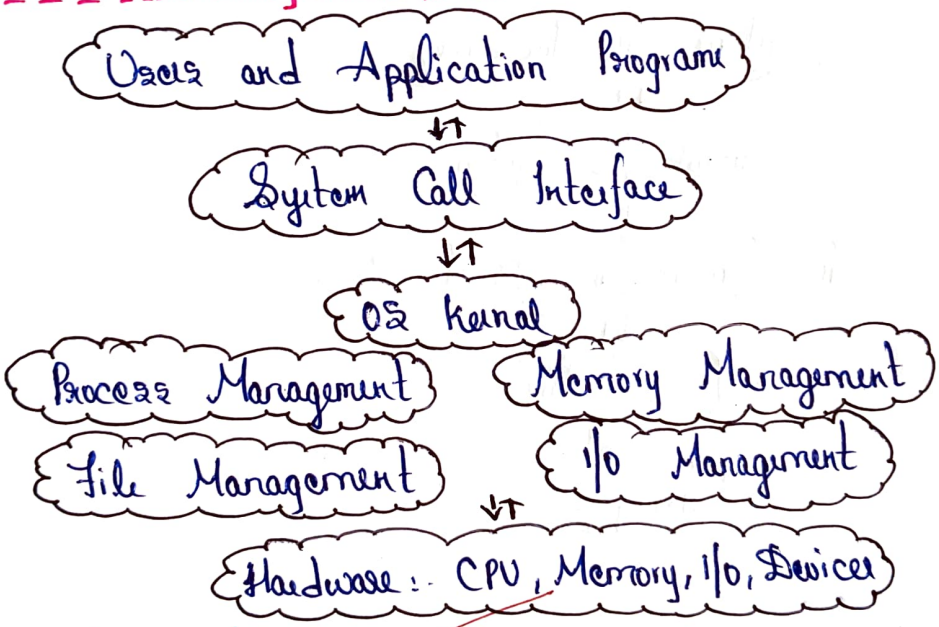
(ii) Multiprocessing OS - Uses multiple CPUs or cores to execute processes

(iii) Multitasking OS - Executes multiple processes efficiently

(iv) Time-Sharing OS:- Allocates CPU time to each user (or) execution.

(v) Distributed OS :- Manages resource across interconnected computers.

(b) Structure of an Operating System



(c) System calls to create and execute a process

In UNIX/Linux, the sequence is

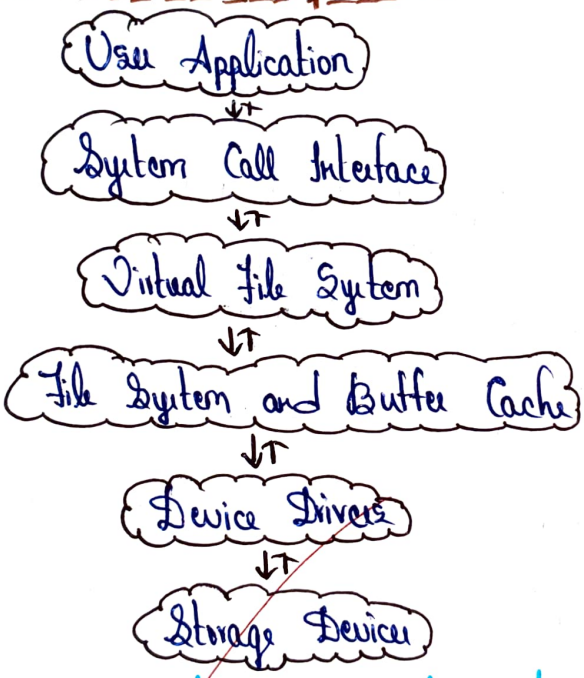
1. fork() - Creates a child process
2. Exec() - Loads and executes a new program in the child
3. wait() - Allows the parent to wait for the child to finish
4. Exit() - Terminates the process.

2) File and Device Management

(a) Categories of System calls

1. File Management - open(), read(), write(), close(), unlink()
2. Device Management - ioctl(), read(), write(), open(), close()
3. Process Management - fork(), exec(), exit(), wait()
4. Information maintenance - stat(), getpid(), time()
5. Communication - pipe(), socket(), send(), recv()

(b) Layered structure for file operations



(c) System call sequence to open, read and close a file

```
#include <fcntl.h>
#include <unistd.h>
int main() {
        char buffer[100];
        int fd = open("data.txt", O_RDONLY);
        if (fd >= 0){
            write (1, buffer, n>0? n :0;
            close (fd);
}       }
```

Real-Time Operating System

(a) Type of OS and Suitability

A Real-Time Operating Systems (RTOS) is suitable for controlling industrial machinery because it processes events within predictable time limits.

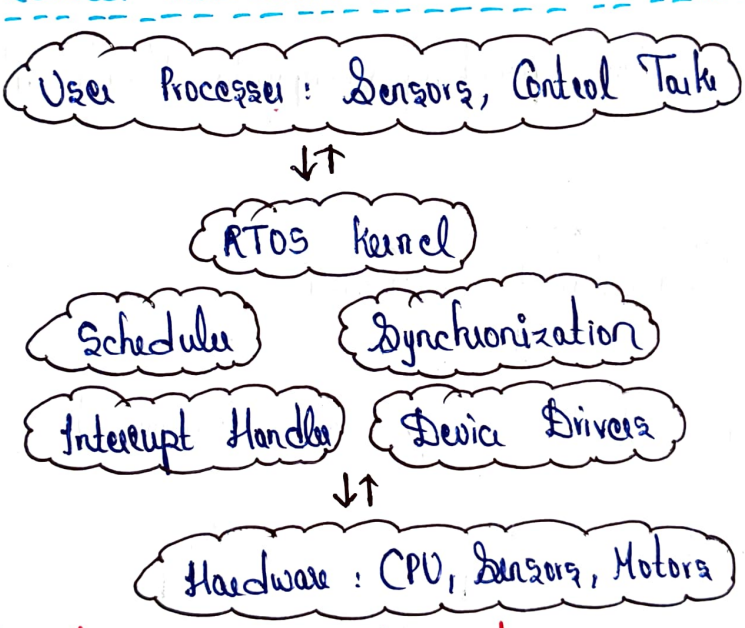
Hard RTOS:- Missing a deadline can cause system failure.

Example:- aircraft control

Soft RTOS:- Occasional missed deadline are tolerable.

Example:- Multi-media Systems

(b) Interaction between hardware, kernal and User process



(c) System calls for process synchronization

1. sem_wait() – Decreases a semaphore value (or) blocks the process
2. sem_post() – Increase the semaphore value and may wake process
3. pthread_mutex_lock() – Locks the mutex to protect shared data
4. pthread_mutex_unlock() – Releases the mutex
5. pthread_cond_wait() – Waits for a condition to become true.

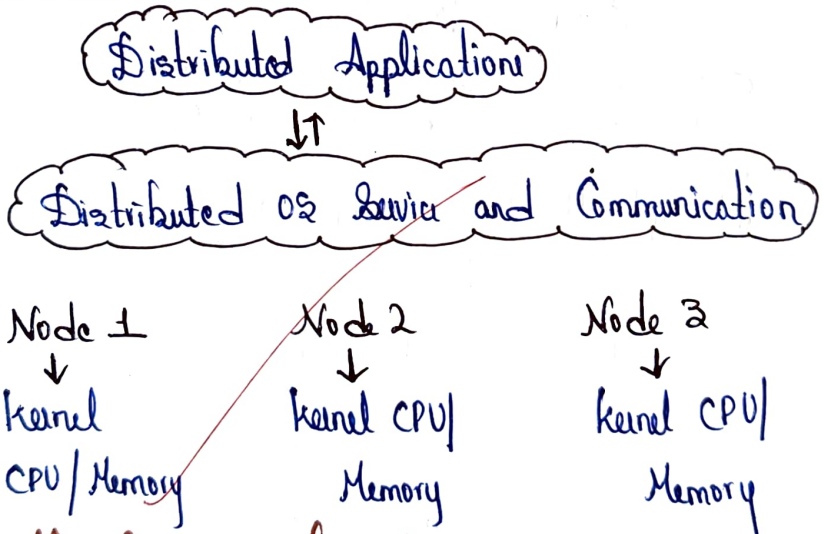
4) Distributed Operating System

(a) Classification and comparision

A Distributed Operating System manages multiple inter connected computers and co-ordinates their resources so they can work together.

Feature	Distributed OS	Centralized OS
Processing	Multiple Computers	One Central Computer
Resource Sharing	Across network nodes	Within a central system
Reliability	Can tolerate some node failure	Central failure may effect
Scalability	Can expand by adding nodes	Mainly internal communication

(b) Architecture of a distributed OS



(c) System call Sequence for IPC

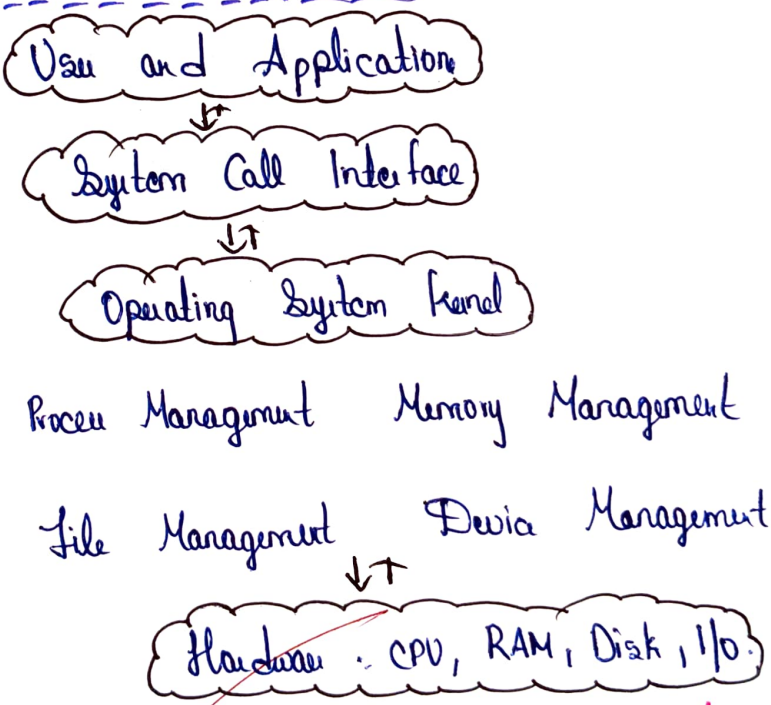
1. pipe() – Creates a communication channel.
2. fork() – Creates a child process
3. write() – Sends data through the pipe
4. read() – Receives data from the pipe.

Major functions and Components of OS

(a) Major functions of an OS

1. Process Management – Creates processes, schedules CPU time, and process
2. Memory Management – Allocates and releases memory and virtual memory
3. Device Management – Controls I/O devices and handles interrupts
4. File Management – Creates, reads, writes and deletes files.
5. Security and protection – Prevents unauthorized access to resources.
6. Resource Allocation – Distributes CPU, memory, and devices.

(b) Block diagram of OS Components :-



(c) System call example for Memory and Device Management

- Mmap() – Maps memory into a process address space
- munmap() – Removes a memory mapping
- brk|sbrk() – Adjusting the process heap boundary on Systems that support them.